구조 논의용

# aicx-callbot

# 구조 한 장으로

콜봇이 어떻게 짜여있는지를 4가지 핵심으로 정리. **큰 그림에 집중하고, 클래스 이름·파일 경로 같은 디테일은 디테일 박스에만**.

① 누구·무엇 ② 두 가지 일하는 방식 ③ 통화 흐름 ④ 코드 구조 원칙

상세 분석은 VOX_AGENT_STRUCTURE.pdf 참조

# 한 줄 요약

> 한 통화에 **봇 여러 명**이 협업한다. 그중 한 명은 **메인**(통화를 시작·자유 응대), 나머지는 **서브**(환불·결제 같은 절차 전문). 메인이 의도를 듣고 적절한 서브에게 넘기고, 서브가 끝나면 다시 메인으로 돌아온다.

## 4가지 핵심

### 1 누구·무엇

한 통화 = **봇 묶음**. 메인 1명 + 서브 N명. 각 봇은 자기만의 페르소나·지식·도구를 가짐.

### 2 두 가지 방식

자유 대화를 잘하는 **프롬프트 봇**과, 절차를 빠짐없이 수행하는 **플로우 봇**. 한 봇은 둘 중 한 방식만.

### 3 통화 흐름

매 발화마다 **2단계** — ① 공통 규칙 검사 → ② 현재 활성 봇 실행. 핸드오프도 봇 실행 결과로 처리. 변수는 모두가 공유.

### 4 코드 원칙

비즈니스 규칙은 한 곳에만. DB·외부 API 사용 부분과 명확히 분리.

## 이후 페이지 구성

§1 누구·무엇 / §2 두 가지 일하는 방식 / §3 분기 필요·불필요 / §4 통화 흐름 / §5 코드 구조 원칙 / §6 구체 시나리오 / §7 현재 설계안 / §8 논의 필요 항목

# 1 누구·무엇

한 통화 안에는 봇 한 명이 아니라 **여러 명이 협업**한다. 그리고 각 봇은 자기만의 자산(페르소나·스킬·지식·도구)을 가진다.

> 💡 한 통화 = 봇 묶음. 봇 한 명 = 페르소나 · 스킬 · 지식 자료 · 도구 4가지로 구성. 통화 단위 설정(음성·인사말 등)은 통화 자체에 붙어있어 한 통화 동안 일관됨.

## 2 두 가지 일하는 방식

봇이 사용자 발화에 응답하는 방식은 두 가지. 한 봇은 둘 중 **하나만** 선택한다 (섞지 않음).

### 프롬프트 봇 — 자유 대화 잘함

**어떻게 동작하나**: 봇에게 "이런 일을 하는 친절한 안내원이야"라고 글로 적어두면, 사용자 발화에 LLM이 알아서 자연스럽게 답함.

**잘 맞는 경우**: 인사·FAQ·일반 응대·의도 파악. 어떤 질문이 올지 미리 모를 때.

**약점**: "이 3가지 정보는 무조건 받아라"같은 절차 강제가 어려움. LLM이 가끔 누락.

### 플로우 봇 — 절차를 빠짐없이

**어떻게 동작하나**: 봇의 응답을 노드로 그래프처럼 그려둠. "1단계: 이름 받기 → 2단계: 사유 받기 → 3단계: 확정". 사용자 발화가 와도 정해진 흐름대로만 진행.

**잘 맞는 경우**: 결제·본인확인·환불처럼 필드 누락이 사고로 이어지는 절차.

**약점**: 만드는 데 시간 들고, 사용자가 예상 밖 말을 하면 어색해질 수 있음.

### 핵심 패턴 — 메인은 프롬프트, 분기만 플로우

> 💡 첫 진입은 의도가 미정이라 **프롬프트가 자연스럽다**. 절차 필요한 분기만 **플로우로 만들어 누락 방지**. 분기가 끝나면 메인으로 돌아옴.

## 3 분기가 필요할 때 vs 필요 없을 때

서브 봇(분기)이 **없어도 콜봇은 잘 동작**한다. 단순 안내·FAQ 봇은 메인 하나로 충분. 분기는 절차가 명시적으로 필요할 때만 추가.

### 경우 1 — 분기가 아예 없는 단순 콜봇

예: 단순 안내봇, FAQ 봇, 인사 봇. 한 통화 묶음 안에 메인 봇만 두면 됨. 모든 발화에 메인 봇이 응대하고 종료. → 빌더가 분기를 만들지 않으면 서브 봇은 0개, 시스템은 그대로 동작.

### 경우 2 — 분기는 있는데 이번 발화가 트리거 안 됨

**예 발화**: "위치가 어디예요?" → 공통 규칙 매치 없음 → 메인 봇이 자기 지식 자료(위치 안내)로 답변 생성 → "강남역 3번 출구에서 도보 5분입니다". **서브 봇은 건드리지도 않음.** 트리거 매치되는 발화가 올 때까지 메인이 계속 응대.

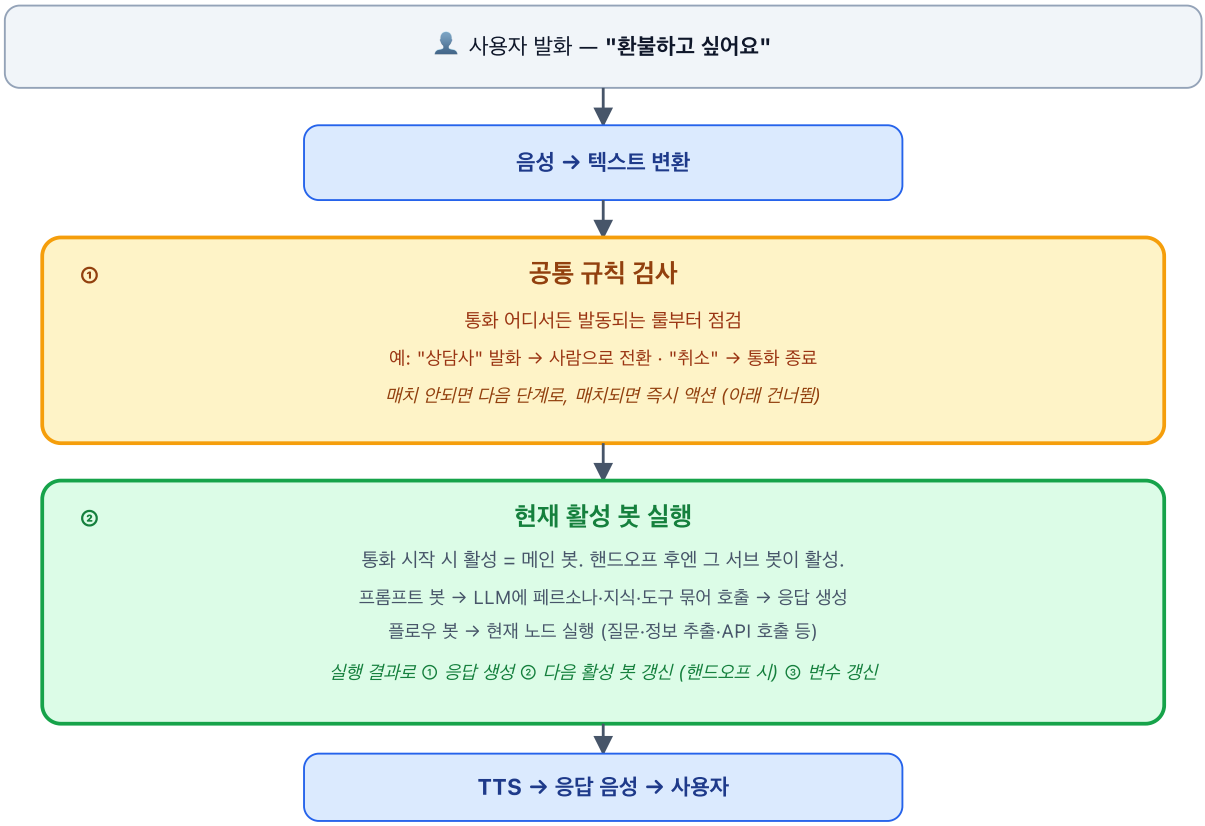
### 경우 3 — 분기 트리거 매치 (메인 → 서브)

**예 발화**: "환불하고 싶어요" → 메인 봇 의도 분류 = "환불" → 환불 봇으로 핸드오프 (변수도 그대로 인계) → 환불 봇이 절차 진행 (주문번호 → 사유 → 환불 API → 확인 SMS) → 환불 봇 종료 → 메인 봇으로 복귀 → 다음 발화 대기.

💡 메인 봇이 **기본 응답자**. 트리거 안 맞으면 메인이 계속 함. 필요할 때만 서브가 잠깐 끼어들어 절차 처리하고 다시 메인으로. **분기 없이 메인 하나만 두는 것도 완전히 OK.**

# 4 통화 흐름 — 발화 한 번이 처리되는 2단계

사용자 발화 한 번이 시스템 안에서 거치는 2단계. **매 발화마다 ① 공통 규칙 검사가 항상 첫 단계**이고, ② 봇 실행 결과에 따라 다음 활성 봇이 갱신된다.




## 핸드오프는 별도 단계가 아님

"메인 봇 → 환불 봇" 전환은 ② **봇 실행 결과의 일부**로 처리된다. 메인 봇 LLM이 응답할 때 "환불 봇으로 넘기자"는 신호를 같이 내면, 다음 활성 봇이 환불 봇으로 갱신될 뿐. **별도 라우팅 서비스 없음.**

> **내부 명칭 (개발자용)**: ① = `Dispatcher` · ② = `Executor` . `Skill.is_frontdoor` 는 "이 스킬이 메인 진입점"이라는 플래그일 뿐, 별도 서비스 아님. 핸드오프는 봇 실행 결과의 후처리 (활성 봇 ID 갱신 + 변수 인계)로 단순화.

## 변수 컨테이너 — 통화 내내 공유되는 메모

2단계 내내 **같은 변수 묶음**이 공유된다. 메인 봇이 받은 이름을 환불 봇이 그대로 쓰고, 환불 봇이 채운 주문번호를 다음 봇이 참조 가능. 핸드오프해도 변수는 끊기지 않음. 변수 출처는 3가지:

- **① 통화 시작 시 주입** — SDK·웹훅·아웃바운드 캠페인이 미리 넣어둠 (customer_name, phone 등)
- **② 시스템 자동** — 통화 ID·시작 시각·발신 번호 등 시스템이 자동 채움
- **③ 대화 중 추출** — 플로우 봇이 사용자 발화에서 뽑아냄 (주문번호·환불 사유·날짜·시간 등)

# 5 코드 구조 원칙 — 비즈니스 규칙은 한 곳에

코드를 4개 층으로 나눈다. 비즈니스 규칙은 가장 안쪽 층에만 두고, 바깥 층은 DB·외부 API 같은 도구를 다룬다.

## 왜 이렇게 나누나

비즈니스 규칙이 DB 모델이나 API 응답 형태에 섞이면, **규칙 하나 바꾸는데 세 군데를 동기화**해야 한다. 안쪽에만 두면 DB 바꾸든 LLM 바꾸든 비즈니스 규칙은 그대로 유지.

> 🚨 절대 규칙: **비즈니스 규칙(예: "메인 봇은 1명만") 절대 DB 모델이나 API 응답 형태에 박지 말 것.** 가장 안쪽 층에만 둔다. 새 기능 만들 때 항상 ① 규칙 정의 → ② 추상 인터페이스 → ③ DB·외부 호출 구현 → ④ 흐름 조립 → ⑤ API 노출 순서.

**내부 명칭 (개발자용)**: 가장 안쪽 = domain · 외부 시스템 연결 = infrastructure · 협업 흐름 = application · 외부 노출 = api. 각 층은 자기보다 안쪽에만 의존한다.

# 구체 시나리오 — △△치과 예약 통화 한 통

앞 페이지의 모든 개념이 한 통화에서 어떻게 맞물리는지.

**Turn 1 — 👤 "여보세요"**
메인 봇이 응대 → 🤖 "안녕하세요, △△치과입니다. 무엇을 도와드릴까요?"

**Turn 2 — 👤 "내일 예약하려고요"**
의도="예약" 분류 → 한 줄 요약 생성 + 변수 인계 → **예약 봇으로 핸드오프 (서브는 깨끗한 history로 시작)**

**Turn 3 — 🤖 예약 봇 진입 → "네, 날짜를 알려주세요"**
현재 단계: 날짜 묻기 → 사용자 발화 대기

**Turn 4 — 👤 "내일 3시요"**
날짜·시간 추출 → 변수 저장 → 다음 단계: 증상 묻기

**Turn 5 — 👤 "충치요"**
증상 추출 → 예약 등록 API → 성공 → 확인 문자 발송

**Turn 6 — 🤖 "예약 완료. 확인 문자 보내드렸어요. 더 도와드릴 게 있나요?"**
예약 봇 end → 멤버십 정책 = return_to_main → 메인 복귀, 예약봇 대화는 메인 history에 합쳐짐

**Turn 7 — 👤 "가격은요?" → 🤖 (가격표 검색) "충치 치료는 평균 X원입니다"**
메인 봇이 지식 자료 검색 후 응답 → 통화 종료

> 💡 한 통화 안에서 메인↔서브가 자연스럽게 두 번 바뀌었지만 사용자는 모드 차이를 전혀 느끼지 못함. 변수(이름·전화)는 끝까지 한 묶음.

# 현재 설계안 — 기본 전제

앞 페이지의 구조에서 도출한 **설계안**. PM 검토 후 합의되면 확정. 이견 있으면 다음 페이지의 핵심 논의 항목과 함께 검토.

| 항목 | 제안 |
|---|---|
| 최상위 단위 | 한 통화 (메인+서브 봇 묶음) — 통화 단위로 일관된 음성·인사말 유지 |
| 멤버 구조 | 메인 1명 + 서브 N명 (필요 시 서브 0개도 OK · 단순 봇 케이스) |
| 음성·인사말 위치 | 통화 단위에. 특정 서브만 예외적으로 음성 덮어쓰기 허용 (외국어 분기 등) |
| 서브 재사용 | 같은 서브 봇을 여러 통화 묶음에 공유 가능 (예: 공통 결제 봇) |
| 모드 혼용 | 한 봇 안에서는 프롬프트/플로우 섞지 않음 — 봇 단위로 모드 분리 |
| 코드 구조 | 비즈니스 규칙은 한 층에만 (DB·API 형식에 섞지 않음) |
| 핸드오프 시 변수 | 변수 컨텍스트는 통화 단위 1개 객체 — 자동 공유, 별도 인계 불필요 |
| 핸드오프 시 페르소나 | 서브 봇의 시스템 프롬프트로 교체. 메인의 페르소나·지식·도구는 잠시 비활성 |
| **변수 namespace** | Skill별 prefix 강제 (예: `reservation.date`, `refund.reason`) — 충돌 방지 |
| **Tool 권한 경계** | Tool은 Tenant(고객사) 단위로 묶임. Bot은 `allowed_tool_ids` 화이트리스트로 사용 가능 도구 제한 |
| **Skill 버전 핀** | Bot이 특정 Skill 버전 참조 (예: `결제봇@v2`). 자동 업그레이드 없음. 명시적 업그레이드만 새 버전 적용 |
| **관측성 (기본 활성)** | turn-level tracing 자동 수집 — latency·cost·노드 ID·LLM 토큰. 운영팀이 통화 흐름을 노드 단위로 추적 가능 |
| **음성 파이프라인** | 이 문서는 에이전트 구조 한정. STT·TTS·VAD·barge-in·AEC 등 오디오 파이프라인은 별도 인프라 설계서 |

💡 위 13개 전제 + 다음 페이지 6가지 결정이 합의되면 설계 확정. 큰 그림은 단순 — **한 통화에 메인+서브 봇 묶음 / 봇은 4가지 자산 / 발화는 2단계 처리 / 비즈니스 규칙은 한 층에만**.

# 논의 필요 — PM 의견 받기

아래 6가지가 합의되면 설계 확정. 각 항목마다 추천안 표시.

### 🔴 분기 이탈 정책

**상황**: 환불 봇 진행 중 "아 그게 아니고 결제 변경"이라고 말함.

A. 무조건 메인 복귀 후 재라우팅
B. 다른 서브로 직접 점프 허용
C. 진행 중인 절차 끝까지 강제 (이탈 금지)

### 🔴 공통 규칙 우선순위

**상황**: 통화 단위 공통 규칙("상담사")과 봇 안 공통 규칙이 동시 매치.

A. 좁은 범위 우선 (봇 단위)
B. 넓은 범위 우선 (통화 단위)
C. 명시적 우선순위 필드

### 🔴 서브 봇 종료 시 동작

**상황**: 환불 봇이 절차 완료하고 end 도달.

A. 자동 메인 복귀 / B. 통화 자동 종료 / C. 명시적 신호 (현재 코드)
**D. 멤버십에 정책 설정 (추천)**

D 예시: 환불봇→return_to_main, 취소봇→end_call, 설문봇→transfer_to:확인봇. 기본값 자동 메인 복귀.

### 🔴 핸드오프 시 대화 history

**상황**: 메인이 사용자와 대화 중 환불봇으로 핸드오프.

A. 전체 history 그대로
B. 비우고 한 줄 컨텍스트만
**C. 메인이 요약해 전달 (추천)**

복귀 시엔 서브 대화를 메인 history에 합쳐 자연스러운 후속 대화.

### 🔴 의도 중첩 처리

**상황**: "예약 취소하고 환불도 되나요?" — 한 발화에 두 의도.

**B. 명확화 질문 (MVP 추천)**
C. 분해 + 순차 처리 (Phase 2)
D. 우선순위 + 후속 확인

B: 메인이 "어느 것부터?" 되묻기. 안전 100%. 운영하며 빈도 보고 C/D로 진화.

### 🔴 고객사(Tenant) 레벨 설정 범위

**상황**: 영업시간·금칙어·상담사 연결 정책이 봇마다 같음.

A. Tenant 레벨로 올림 (봇들이 자동 상속)
B. 봇 레벨 유지 (자유도↑ 중복↑)
**C. 영업시간·금칙어=Tenant, voice·greeting=통화 단위 (분리, 추천)**